AI Assignment: Reasoning Agent

Dishant Kaushik | Roll No. M24MA2003

1. Overview

The task is to decide whether a query is entailed by a knowledge base of facts and rules. For the evaluation the knowledge base and the query are given directly as logic, so the main input of my system is a set of formulas. English input is still supported through two parsers, because the public datasets are in English. Every True/False answer is produced by a classical procedure that I implemented: forward chaining, backward chaining, resolution with unification, and forward propagation through causal equations. No language model is used at any point.

The agent looks at each problem and picks the reasoner:

Input	Representation	Reasoner
counterfactual question over a causal model (English or equations)	boolean structural equations	abduction, intervention, forward propagation
logic, Horn theory, literal query	facts + rules with negation	forward chaining, closed world
logic, anything else	clauses	resolution refutation, best-first
English, every sentence fits the simple grammar	facts + rules with negation	forward chaining, closed world
other English	first order formulas -> clauses	resolution refutation, best-first

Table 1: how a problem is routed (checked from top to bottom)

The task asks about entailment, so all labels are reduced to two classes: 'not entailment' (RuleTaker), 'False' and 'Uncertain' (FOLIO) and 'no' (CounterBench) all become False.

2. Representation

2.1 Logic input

The instructor did not fix a notation, so the parser accepts the common ones: FOLIO style Unicode (∀x (Young(x) ∧ ¬Kind(x) → Smart(x))), ASCII (`forall x (young(x) & ~kind(x) -> smart(x))`, with not / and / or / ! / - / | / => / <-> / xor) and Prolog clauses (`smart(X) :- young(X), \+ kind(X).`). ProofWriter's triple form (`("Bob" "is" "big" "+")`, rules as `((...) -> (...))`) is read as well. A knowledge base can be one formula per line, formulas separated by ';' or '. ', a JSON list, or lines with numbers or labels in front (*1.*, *P2:*, *Rule 3:*). Free variables x, y, z, u, v, w are read as universally quantified. In Prolog clauses capitalised arguments are variables. Formulas are stored as nested Python tuples such as `('all', '?x', ('imp', ('atom', 'young', ('?x',)), ('atom', 'smart', ('?x',))))`. Predicate names are compared in lower case. FOLIO's annotations are not always consistent about capital letters (Writtenby / WrittenBy), and matching on the exact spelling cost about half a percent there.

2.2 Facts and rules

When a theory is Horn (ground literals, and *for all x: conjunction of literals -> literal*, with negation allowed anywhere) it is converted to the representation of my rule engine. An atom is a tuple like `('chase', 'cat', 'bald_eagle')`, a literal is an atom with a sign, and a rule is a list of body literals and one head literal. The same structure comes out of my pattern parser for RuleTaker English (*If someone is young then they are smart*, *All red, round things are big*, *The cat does not chase the dog*, ...).

2.3 Causal models

A causal model is a set of boolean equations, one per non-root variable. It can be given as equations (Vork = Blaf ∧ Trune, Blaf ↔ ¬Ziklo, a bare literal is an observation) or as CounterBench English (*A and B together cause not C*, *We observed X*). The question stays in English: *Would Y occur if not X instead of X?*, *... if not X and not Z?* and the nested *Assume not X, ... further suppose not Z.*

2.4 English fallback for FOLIO style text

spaCy (small English model) gives lemmas, POS tags and the sentence subject. On top of that I wrote rules for quantifiers, generic statements, if-then, if and only if, either-or, neither-nor, relative clauses and pronouns. A predicate name is the sorted list of stemmed content words of its phrase. On this route I also add word subset axioms: if the words of predicate B are a strict subset of those of A, then A(x) -> B(x).

3. Reasoning algorithms

3.1 Forward chaining with negation as failure

The model of a theory is computed once and every query about that theory is answered from it. Rule bodies are matched against the facts with unification, positive literals first, using an index on the predicate name, until no new fact appears. Under the closed world assumption *not X* holds when X cannot be derived. That is only sound if X is completely known before it is tested, so the rules are stratified: every predicate gets a level higher than the predicates it depends on negatively, and the levels run in order. About 10% of the RuleTaker theories have a cycle through negation at the predicate level. For those the rules are grounded over the constants and the well-founded model is computed with the alternating fixpoint. An over-estimate and an under-estimate of the true atoms are refined in turn until they stop changing.

3.2 Backward chaining

A goal-driven version: depth-first SLD resolution with negation as failure, a loop check (a goal already on the current path fails), and a memo of proved goals. It gives the same answers as forward chaining and is what prints the proof trees in section 5.

3.3 Resolution

Formulas are put in clause form: remove implications, iff and xor, push negations inward, rename variables apart, Skolemise, and distribute OR over AND with a size limit. To prove a goal, its negation is added and the prover looks for the empty clause. The search uses the set of support strategy, and the next clause to expand is always the one with the lowest cost (number of literals plus term depth). That makes it a uniform cost / best-first search over derived clauses. Variants are detected by renaming variables to a canonical form, tautologies are dropped, and unification has the occurs check. A proof attempt stops after 500 derived clauses, with a 2 second time cap only as a safety net. A clause limit gives the same answer on any machine, which a pure time limit does not. Equality gets reflexivity, symmetry and unique names for constants. The answer is True only if the goal is proved and its negation is not. If a premise does not parse it is skipped, so one bad line does not lose the whole problem. Two common slips are also repaired: a quantifier with no variable and the letter v used for 'or'.

3.4 Counterfactuals

Abduction, action, prediction. All assignments of the root variables are enumerated (a small search, there are only a few roots), and the ones consistent with the observations are kept, together with X being true for *instead of X* questions. Then the intervened variables are set and the values are propagated forward in topological order (iterated to a fixpoint for the five cyclic models). Y is entailed if it is true in every consistent world.

3.5 Closed world or open world?

RuleTaker labels follow the closed world assumption and FOLIO labels the open world one, and once both arrive as logic the reasoner has to choose. I answered the Horn problems from both datasets both ways (Table 2). Open world answers get a quarter of the RuleTaker questions wrong: 537 negated queries like ¬*Nice(bob)*, and 76 positive ones whose proof needs a rule with a negated condition. Closed world costs 3 FOLIO train problems and none on validation. So Horn theories with a literal query go to forward chaining, and everything else goes to resolution.

Horn problems	count	closed world correct	open world correct
RuleTaker test sample (300 per config)	2400	2400	1787
FOLIO train (gold FOL)	132	117	120
FOLIO validation (gold FOL)	20	20	20

Table 2: the same Horn problems answered with each semantics

4. Experiments on the public data

Timings are on my laptop (Windows 11, Python 3.13, single thread). RuleTaker and CounterBench only exist in English, so to test logic input I converted them: RuleTaker theories through my parser into the three notations, and CounterBench models into equations. FOLIO has its own FOL annotations. I developed on RuleTaker train/dev and FOLIO train. RuleTaker test is fully held out. CounterBench has no split, so its numbers are on the same data I developed with.

4.1 RuleTaker

input	accuracy (%)	time for 151,911 questions (s)
logic, Unicode	100.00	7.8
logic, ASCII	100.00	7.9
logic, Prolog	100.00	8.1
English, naive forward chaining	99.95 (76 errors)	4.3
English, stratified forward chaining	100.00	4.4
English, backward chaining	100.00	4.5

Table 3: RuleTaker test split (times include parsing)

The 76 errors of the naive version (all rules in one fixpoint) are rules whose negated condition was checked before the atom had been derived, and stratification removes them. Getting the English parser to 100% took several rounds on dev: 79.85% for the first version, 93.72% once multi-word names like *the bald eagle* were split correctly, 98.75% with *Red things are big* read as a rule, 99.976% after tightening objects, and 100% with grounded stratification plus the well-founded semantics.

4.2 FOLIO with its FOL annotations

setting	train binary	val binary	val 3-way	val time (s)
always False (baseline)	61.24	64.53	-	-
resolution	81.52	84.73	73.89	8.3
resolution + word subset axioms	81.62	83.74	72.91	8.8
final: Horn -> forward chaining, else resolution	81.22	84.73	-	4.3

Table 4: FOLIO, accuracy in %. 1001 train and 203 validation problems.

The word subset axioms gained one problem on train and lost two on validation, so I do not use them on logic input. I chose the search limit on train. Limits of 0.25 to 2 seconds, and 500 or 1000 clauses, all gave the same accuracy, and 200 clauses lost two problems. With 500 clauses the train set runs in 15 s instead of 62 s with the earlier 1 second limit. Skipping unparseable premises recovered 3 train problems. Going through the true problems the prover misses, most of the misses come from the annotations rather than from the search. A 10 second limit with 30,000 clauses did not prove a single extra true problem on validation. Typical causes are a constant spelled 'bonne' instead of 'bonnie', `FemaleTennisPlayer(x)` in one premise but Female(x) ∧ TennisPlayer(x) in another, or a free variable where a constant was meant. The final setting is also faster, because the Horn problems skip the prover.

4.3 CounterBench

type	basic	conditional	joint	nested	backdoor	all
correct	250/250	230/250	240/250	240/250	180/200	1140/1200
accuracy (%)	100.0	92.0	96.0	96.0	90.0	95.0

Table 5: CounterBench. English and equation input give the same answer on all 1200 questions (0.11 s and 0.29 s).

The 60 errors come in blocks of 5 or 10 from a few graph templates. For some of them no reading I could think of gives the gold label. For example, the intervention *not Vork* with *Lumbo = Vork* is labelled as Lumbo occurring. I tried two other readings of the nested questions and an 'override' reading. Each fixed some of these cases and broke more others, so I kept the standard semantics.

4.4 English fallback on FOLIO

From English the result is much lower: 67.49% on validation in all three settings (parser only, + word subset axioms, + check for contradictory premises) against a 64.53% baseline. This route keeps a bigger search budget (3000 clauses, 1 s), because with 500 clauses it lost one validation problem. On train it is 67.93%, 69.53% and 69.63% against a 61.24% baseline. In about half of the missed true problems the conclusion uses a predicate that never occurs in the premises, which needs relations and world knowledge my parser does not have. I tuned this parser on train, and looked at validation errors once (two fixes after that).

4.5 Final run

The last notebook cell builds a public evaluation file in logic form: 1000 RuleTaker test questions per config (8000) in Unicode FOL, the 203 FOLIO validation problems with their FOL, and the 1200 CounterBench questions with equation models. It clears all caches and predicts everything:

	value
queries	9403
total inference time	7.66 s (0.81 ms per query)
accuracy, all	99.03 %
RuleTaker / CounterBench / FOLIO	100.00 / 95.00 / 84.73 %

Table 6: final run of the notebook

RuleTaker needs about 0.05 ms per question in logic form and CounterBench about 0.2 ms. FOLIO takes most of the rest, about 5 s for 203 problems. Before the clause limit, the same run took 22.25 s with the same accuracy.

When TEST_FILE is set, the notebook skips the public-data experiments and only runs the test file. spaCy is then not loaded either, so it does not even need to be installed for logic input. I checked this on a copy of the submission folder with no data folder. An 800 problem test file in JSONL with other column names and 0/1 labels ran through the whole notebook in under 4 seconds.

5. Reasoning traces

Copied from the notebook output (section 8). Logic symbols are written in ASCII here (~ not, & and, | or, ->) because the PDF font does not have them.

5.1 Example from the assignment sheet, given as logic (backward chaining proof tree)

```
Young(bob)
Kind(bob)
forall x (Young(x) -> Smart(x))
forall x (Smart(x) & Kind(x) -> Successful(x))
successful(bob)   [rule: smart(?x) & kind(?x) -> successful(?x)]
    smart(bob)   [rule: young(?x) -> smart(?x)]
        young(bob)   [given fact]
    kind(bob)   [given fact]
```

5.2 RuleTaker depth-3 question that needs negation as failure

```
The bear is green. The cat chases the dog. The dog eats the bear. The rabbit eats the bear.
If something does not chase the bear then it is green. ... Green things are red.
If something is red then it eats the rabbit.
Q: The cat eats the rabbit.
eat(cat, rabbit)   [rule: If something is red then it eats the rabbit]
    red(cat)   [rule: green things are red]
        green(cat)   [rule: If something does not chase the bear then it is green]
            not chase(cat, bear)   [cannot be proved, negation as failure]
```

5.3 FOLIO validation problem in logic, resolution refutation

```
forall x (WildTurkey(x) -> (EasternWildTurkey(x) | OsceolaWildTurkey(x) | GouldsWildTurkey(x) |
                            MerriamsWildTurkey(x) | RiograndeWildTurkey(x) | OcellatedWildTurkey(x)))
~(EasternWildTurkey(tom))     ~(OsceolaWildTurkey(tom))     ~(GouldsWildTurkey(tom))
~(MerriamsWildTurkey(tom) | RiograndeWildTurkey(tom))       WildTurkey(tom)
goal: OcellatedWildTurkey(tom)
not Horn -> resolution refutation. proved: True  given clauses picked: 7
  1. ~ocellatedwildturkey(tom)   (negated goal)
  2. ~wildturkey(?v) | easternwildturkey(?v) | gouldswildturkey(?v) | merriamswildturkey(?v)
     | ocellatedwildturkey(?v) | osceolawildturkey(?v) | riograndewildturkey(?v)   (premise)
  3. ~wildturkey(tom) | easternwildturkey(tom) | gouldswildturkey(tom) | merriamswildturkey(tom)
     | osceolawildturkey(tom) | riograndewildturkey(tom)   (resolvent)
  4. ~easternwildturkey(tom)   (premise)
  5. ~wildturkey(tom) | gouldswildturkey(tom) | merriamswildturkey(tom) | osceolawildturkey(tom)
     | riograndewildturkey(tom)   (resolvent)
  6. ~osceolawildturkey(tom)   (premise)
  7. ~wildturkey(tom) | gouldswildturkey(tom) | merriamswildturkey(tom) | riograndewildturkey(tom)   (resolvent)
  8. ~gouldswildturkey(tom)   (premise)
  9. ~wildturkey(tom) | merriamswildturkey(tom) | riograndewildturkey(tom)   (resolvent)
 10. ~merriamswildturkey(tom)   (premise)
 11. ~wildturkey(tom) | riograndewildturkey(tom)   (resolvent)
 12. ~riograndewildturkey(tom)   (premise)
 13. ~wildturkey(tom)   (resolvent)
 14. wildturkey(tom)   (premise)
 15. []   (empty clause)
```

5.4 CounterBench conditional question

```
We know that Ziklo causes not Blaf, Blaf and Trune together cause Vork, Vork causes Lumbo.
We observed Trune
Would Lumbo occur if not Ziklo instead of Ziklo?  gold: yes
structural equations:
    blaf = not ziklo
    vork = blaf and trune
    lumbo = vork
roots: ['ziklo', 'trune']  known facts: {'trune': True, 'ziklo': True}
actual world:         {'ziklo': 1, 'blaf': 0, 'trune': 1, 'vork': 0, 'lumbo': 0}
after do(ziklo=0): {'ziklo': 0, 'blaf': 1, 'trune': 1, 'vork': 1, 'lumbo': 1}
1 consistent world(s), lumbo = True
```

6. Limitations

The choice between closed and open world is made from the shape of the theory. A Horn problem that was written with open world labels and asks a negated question will be answered wrongly, which is the 3 FOLIO train cases in Table 2. If the private set is mostly of that kind, the rule would have to change. On logic input most remaining FOLIO errors come from the annotations, which my system cannot repair. The English routes are much weaker than the logic route for FOLIO. The remaining CounterBench errors look like quirks of a few templates, and I did not fit them.